

| 第九周作业 -1- 基础作业

| 第九周作业 -1- 基础作业

| 1.1 环境搭建与训练验证

1. 确认 Unitree RL Lab 安装成功，运行以下命令列出所有可用任务：

```
cd ~/unitree_rl_lab/ ./unitree_rl_lab.sh -l
```

2. 启动 Unitree-G1-29dof-Velocity 训练：

```
./unitree_rl_lab.sh -t --task Unitree-G1-29dof-Velocity
```

3. 观察训练成功启动后，使用 Ctrl + C 结束训练；

4. 找到本次训练生成的 run 目录，列出其中的文件。

提供以下验证材料：

- 任务列表截图 1 张：能看到 5 个可用任务；
- 训练启动截图 1 张：能看到训练输出和 reward 值；
- run 目录文件列表截图 1 张。

1. 确认 Unitree RL Lab 安装成功：

```
(isaacclab) [root@gpufree-container ~/myLab/unitree_rl_lab]# ls
deploy  docker  logs      pyproject.toml  scripts  unitree_rl_lab.sh
doc     LICENCE  outputs   README.md      source
(isaacclab) [root@gpufree-container ~/myLab/unitree_rl_lab]# ./unitree_rl_lab.sh -l
+-----+-----+-----+-----+-----+
|                                     Available Environments in Unitree RL Lab                                     |
+-----+-----+-----+-----+-----+
| S. No. | Task Name                               | Entry Point                               | Config                                                                 |
+-----+-----+-----+-----+-----+
| 1       | Unitree-G1-29dof-Velocity              | isaacclab.envs:ManagerBasedRLEnv        | locomotion.robots.g1.29dof.velocity_env_cfg:RobotEnvCfg             |
| 2       | Unitree-Go2-Velocity                   | isaacclab.envs:ManagerBasedRLEnv        | locomotion.robots.go2.velocity_env_cfg:RobotEnvCfg                   |
| 3       | Unitree-H1-Velocity                     | isaacclab.envs:ManagerBasedRLEnv        | locomotion.robots.h1.velocity_env_cfg:RobotEnvCfg                     |
| 4       | Unitree-G1-29dof-Mimic-Dance-102       | isaacclab.envs:ManagerBasedRLEnv        | mimic.robots.g1_29dof.dance_102.tracking_env_cfg:RobotEnvCfg         |
| 5       | Unitree-G1-29dof-Mimic-Gangnanm-Style | isaacclab.envs:ManagerBasedRLEnv        | mimic.robots.g1_29dof.gangnanm_style.tracking_env_cfg:RobotEnvCfg    |
+-----+-----+-----+-----+-----+
```

2. 训练启动截图：

```
Terminal 终端 - root@gpufree-container: ~/myLab/unitree_rl_lab
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)

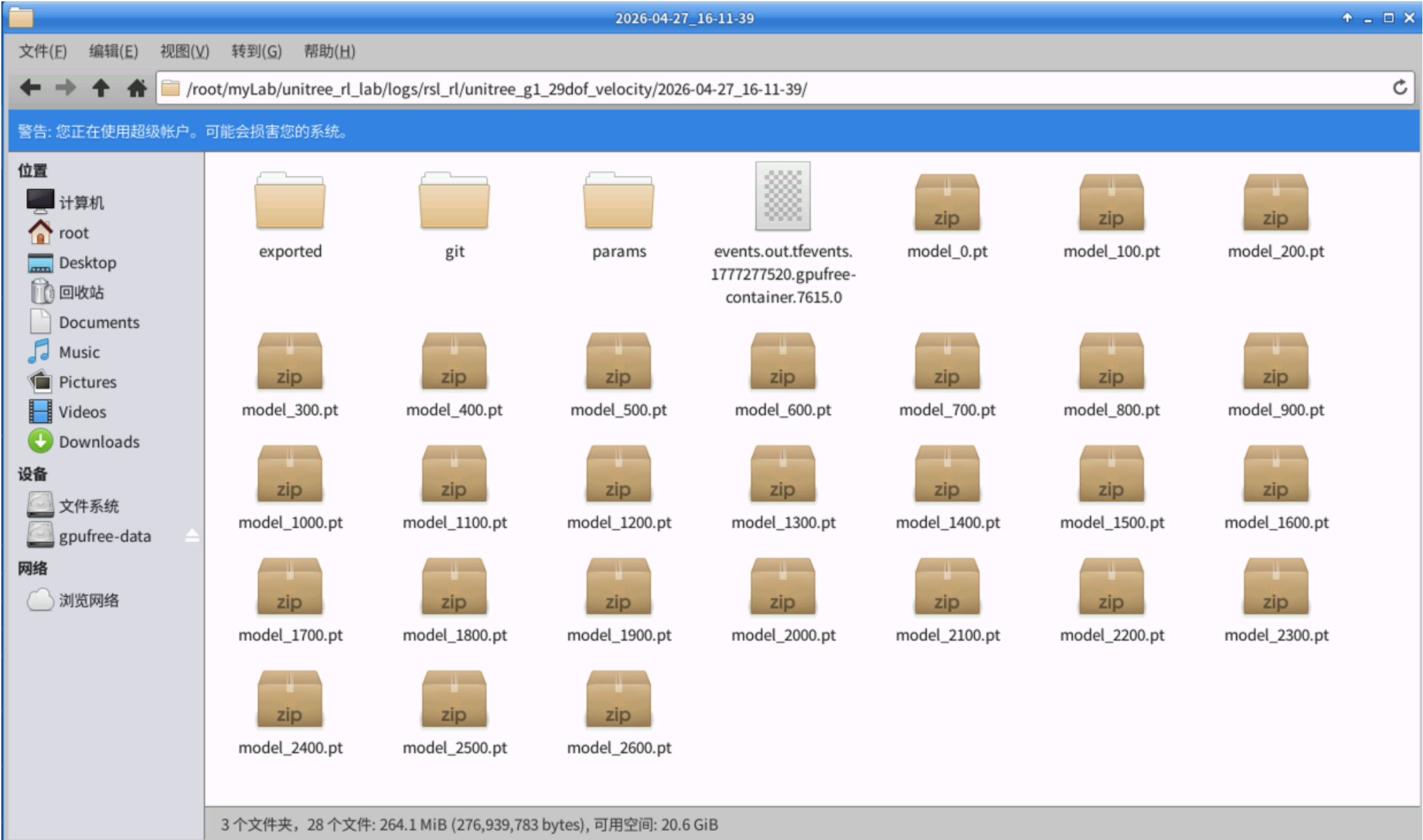
Time elapsed: 00:00:05
ETA: 15:34:15

#####
      Learning iteration 2/50000

      Computation: 52261 steps/s (collection: 1.786s, learning 0.095s)
      Mean action noise std: 0.99
      Mean value_function loss: 0.3720
      Mean surrogate loss: -0.0078
      Mean entropy loss: 40.8712
      Mean reward: -4.77
      Mean episode length: 50.03
      Episode_Reward/track_lin_vel_xy: 0.0266
      Episode_Reward/track_ang_vel_z: 0.0024
      Episode_Reward/alive: 0.0079
      Episode_Reward/base_linear_velocity: -0.0047
      Episode_Reward/base_angular_velocity: -0.0325
      Episode_Reward/joint_vel: -0.0429
      Episode_Reward/joint_acc: -0.0185
      Episode_Reward/action_rate: -0.1525
      Episode_Reward/dof_pos_limits: -0.0011
      Episode_Reward/energy: -0.0010
      Episode_Reward/joint_deviation_arms: -0.0087
      Episode_Reward/joint_deviation_waists: -0.0194
      Episode_Reward/joint_deviation_legs: -0.0234
      Episode_Reward/flat_orientation_l2: -0.0234
      Episode_Reward/base_height: -0.0026
      Episode_Reward/gait: 0.0125
      Episode_Reward/feet_slide: -0.0099
      Episode_Reward/feet_clearance: 0.0495
      Episode_Reward/undesired_contacts: -0.0127
```

1.

3. log 目录文件列表截图：



1.2 训练输出分析

结合训练终端输出、操作手册和课堂内容，回答以下问题：

- 1. 对比第八讲的 `Isaac-Humanoid-v0`，`Unitree-G1-29dof-Velocity` 新增了哪些 reward 项？这些新增项分别面向什么需求？
- 2. 训练输出中有两个 Curriculum 指标：`Curriculum/terrain_levels` 和 `Curriculum/lin_vel_cmd_levels`，它们各自含义是什么？为什么训练初期它们的值都很低？
- 3. 训练初期 `Episode_Termination/bad_orientation` 的值为 1.0，这说明了什么？随着训练推进，这个值应该发生什么变化？

1.2.1 Isaac-Humanoid-v0 和 Unitree-G1-29dof-Velocity reward 对比

- 1. Isaac-Humanoid-v0 的 reward（奖励/惩罚）项及其对应的权重（weight）和实现函数如下。该环境主要奖励机器人的 **存活** 与 **向目标点移动**（`alive`，`progress`，`move_to_target`），同时通过施加较小的惩罚来保证动作的 **平滑**、**节能** 与 **安全性**（`action_l2`，`energy`，`joint_pos_limits`）

Reward 项	权重 (Weight)	对应函数 (Function)	说明（推测）
progress	1.0	progress_reward	目标进度奖励，通常基于距离目标点 (<code>target_pos</code>) 的距离。
alive	2.0	is_alive	存活奖励，只要机器人没有倒地/触发终止条件就能获得的高额固定奖励。
upright	0.1	upright_posture_bonus	直立姿态奖励，鼓励机器人保持直立。
move_to_target	0.5	move_to_target_bonus	向目标移动奖励，可能用于奖励机器人的有效运动方向。
action_l2	-0.01	action_l2	动作 L2 范数惩罚项，抑制输出过大的动作指令，以保证动作平滑。
energy	-0.005	power_consumption	能耗惩罚项，限制总做功，避免多余的高频率/高扭矩抖动。
joint_pos_limits	-0.25	joint_pos_limits_penalty_ratio	关节位置限制惩罚项，避免关节达到或超出物理限位，保护硬件。

- 2. Unitree-G1-29dof-Velocity 的速度控制任务（velocity control）的所有的 reward（奖励和惩罚）项及其权重和功能如下：

- *标跟踪与存活奖励 (正奖励)*

Reward 项	权重 (Weight)	对应函数 (Function)	说明
track_lin_vel_xy	1.0	track_lin_vel_xy_yaw_frame_exp	核心奖励 ：跟踪基座在 xy 平面的目标线速度。
track_ang_vel_z	0.5	track_ang_vel_z_exp	核心奖励 ：跟踪基座绕 Z 轴的目标角速度（转向）。
alive	0.15	is_alive	存活奖励，鼓励机器人保持不倒地。
gait	0.5	feet_gait	步态奖励，鼓励机器人遵循设定的周期性步态（这里周期设为 0.8 秒，双腿相位差 0.5）。
feet_clearance	1.0	foot_clearance_reward	抬腿高度奖励，鼓励机器人在摆动期将脚抬起至目标高度（0.1 米），避免绊倒。

- *运动平滑与节能惩罚 (负惩罚)：*

Reward 项	权重 (Weight)	对应函数 (Function)	说明
base_linear_velocity	-2.0	lin_vel_z_l2	惩罚 Z 轴线速度（避免身体上下过度起伏跳跃）。

Reward 项	权重 (Weight)	对应函数 (Function)	说明
base_angular_velocity	-0.05	ang_vel_xy_l2	惩罚 X/Y 轴的角速度（避免身体前后、左右过度摇晃）。
joint_vel	-0.001	joint_vel_l2	惩罚过大的关节速度，使运动更平滑。
joint_acc	-2.5e-07	joint_acc_l2	惩罚关节加速度，保护电机并减少高频抖动。
action_rate	-0.05	action_rate_l2	动作变化率惩罚，限制相邻两次控制指令之间的变化幅度。
energy	-2.0e-05	energy	能量消耗惩罚，鼓励机器人以更节能的方式运动。

- 姿态约束与硬件保护惩罚 (负惩罚)

Reward 项	权重 (Weight)	对应函数 (Function)	说明
dof_pos_limits	-5.0	joint_pos_limits	强惩罚：防止关节逼近或突破物理限位。
flat_orientation_l2	-5.0	flat_orientation_l2	强惩罚：促使机器人的躯干保持水平，惩罚翻滚 (roll) 和俯仰 (pitch) 角。
base_height	-10.0	base_height_l2	最强惩罚：惩罚偏离目标身高（目标设为 0.78 米）的情况，迫使其保持站立姿态。
feet_slide	-0.2	feet_slide	滑步惩罚，如果在脚部接触地面时仍有水平速度则受到惩罚。
undesired_contacts	-1.0	undesired_contacts	意外碰撞惩罚，即除了脚部 (ankle) 之外的身体其他部位（如小腿、膝盖等）接触地面时给予惩罚。

- 各部位关节偏移惩罚 (负惩罚)

为了让拥有 29 个自由度的 G1 机器人保持默认或自然的参考姿态，对不同部位的关节偏离默认位置（L1 范数）施加了专门的惩罚：

Reward 项	权重 (Weight)	目标关节正则表达	说明
joint_deviation_arms	-0.1	肩、肘、腕关节	弱惩罚，保持手臂贴合身体，避免走路时手臂乱挥。
joint_deviation_waists	-1.0	包含 waist 的关节	中等惩罚，限制腰部过度扭曲。
joint_deviation_legs	-1.0	髋部 Roll 和 Yaw 关节	中等惩罚，鼓励双腿在矢状面上运动，防止出现“外八字”等不规范步态。

1. 对比基础的 **Isaac-Humanoid-v0** (前往目标点任务), **Unitree-G1-29dof-Velocity** (全向速度跟踪任务) 在 Reward 设计上有了质的飞跃。这些新增项主要面向 **真实的物理部署 (Sim-to-Real)**、**高自由度带来的动作冗余** 以及 **更高级的步态质量要求**。以下是 **Unitree-G1** 新增的 Reward 项及其面向的具体需求分类：

- 任务目标的转变：从“位置”到“速度”
 - 弃用： **progress**, **move_to_target** (面向坐标点的移动)。
 - 新增： **track_lin_vel_xy** (XY 线速度跟踪), **track_ang_vel_z** (Z 轴角速度跟踪)。
 - 面向需求： 机器人需要能够接收人类遥控器或上层导航模块发出的 **实时速度指令**（例如“向前走的同时向左转”），而不是单纯地走向一个固定的物理坐标。这也是真实机器狗/人形机器人最标准的底层控制器接口
- 拟人化与步态质量 (Gait & Foot Trajectory)
 - 新增： **gait** (强制周期性相位的步态), **feet_clearance** (抬脚高度), **feet_slide** (防止滑步)。
 - 面向需求：
 - 解决“拖步”与“滑冰”**：在基础强化学习中，机器人极易学会靠双脚在地上滑动（类似于滑冰）来移动，这在真实地面上摩擦力大会直接摔倒。
 - Sim-to-Real**： **feet_clearance** 强制机器人抬腿跨步， **feet_slide** 惩罚落地时的水平速度， **gait** 则强制左右脚交替的节律。这三者结合，保证了机器人能走出真实物理世界中稳健且美观的真实步态。
- 躯干稳定性与平顺度 (Base Stability)
 - 弃用： 仅靠简单的 **upright** 奖励直立。
 - 新增： **base_height** (严格限制身高), **flat_orientation_l2** (躯干水平度), **base_linear_velocity** (惩罚 Z 轴上下起伏), **base_angular_velocity** (惩罚 XY 轴摇晃)。
 - 面向需求：
 - 云台稳定性**：真实机器人头部/胸部通常装有雷达和相机，如果走路时躯干上下剧烈跳跃或左右疯狂摇晃，SLAM 和视觉算法将直接失效。
 - 通过严惩 Z 轴速度和 XY 轴角速度，倒逼神经网络学会利用双腿像汽车悬挂一样吸收冲击，保持“上身不动下身动”的稳定效果。
- 高自由度姿态约束 (High-DoF Regularization)
 - 新增： **joint_deviation_arms**, **joint_deviation_waists**, **joint_deviation_legs** (各部位关节偏离惩罚)。
 - 面向需求：
 - 抑制“奇葩”动作**：G1 有 29 个自由度，如果不加约束，AI 会为了保持平衡学会像大猩猩一样挥舞双臂，或者出现“内八字/外八字”等奇怪姿态。
 - 通过将手臂、腰部、大腿的横向旋转等非核心驱动关节约束在“默认自然姿态”附近，可以强制机器人以符合人类审美的方式行走，避免动作冗余。
- 极致的硬件保护与马达寿命 (Hardware Safety)
 - 新增： **joint_vel** (限制关节速度), **joint_acc** (限制关节加速度), **action_rate** (动作变化率限制), **undesired_contacts** (防摔保护)。
 - 面向需求：
 - 保护减速器与电机**：基础环境仅惩罚力矩大小 (**action_l2** 和 **energy**)，但这无法阻止高频抖动。真实的减速器如果频繁承受极高加速度 (**joint_acc**) 或高频反转指令 (**action_rate**)，会瞬间过热或打齿损坏。
 - undesired_contacts** 则是为了防止机器人学会“膝盖走路”或“手脚并用爬行”这种在仿真里高效但在真实世界里会磨坏机器人的奇葩策略。
- 总结：如果说 **Isaac-Humanoid-v0** 的 Reward 是为了证明 **" 强化学习能让积木人站起来并往前走 "**；那么 **Unitree-G1-29dof-Velocity** 的 Reward 则是典型的 **" 工业级 Sim-to-Real 配置 "**。它的核心诉求不再是走得快，而是 **走得像人、走得平稳、并且绝对不能把真机硬件弄坏**。

I 1.2.2 Curriculum 指标说明

在这两个指标中， **Curriculum**（课程学习）是强化学习中一种极其重要的训练策略，它的核心思想与人类教育非常相似：**由易到难，循序渐进**。如果在训练的第一步，就让一个还不会走路的机器人去跑过碎石路（高目标速度 + 复杂地形），机器人会瞬间摔倒，并且由于摔得太快，它根本无法从环境中收集到有效的学习数据。

因此，训练引入了以下两个课程指标：

- Curriculum/terrain_levels**（地形难度等级）
- 含义**：表示当前机器人所处地形的复杂程度。

- **为何初期很低**：在训练初期，这个值通常是 0。这代表环境强行把所有机器人放置在 **绝对平坦的平面** 上。只有当机器人在平地上学会了基本的站立和行走，且能稳定存活一段时间后，算法才会让它“升级”（Level Up）。升级后，地形会逐渐变成带有微小起伏的坡道、不平整的碎石地，最终演变为高难度的阶梯或断裂带。
2. Curriculum/lin_vel_cmd_levels（线速度指令难度等级）
- **含义**：表示系统向机器人下发的“目标速度指令”的难度/范围。
- **为何初期很低**：在初期，机器人的平衡能力极差。如果立刻给它下发“以 2.0 m/s 的速度狂奔”的指令，它会直接倾倒。因此，在 Level 0 时，系统通常只要求它保持原地踏步（速度指令接近 0 m/s），或者以极慢的速度（如 0.2 m/s）微动。随着它完美达成了这些慢速指标，系统会逐渐放宽指令范围，开始要求它挑战 1.0 m/s 甚至更高的极限速度。
3. **总结**：训练初期这两个值都很低，是因为系统正在“**给婴儿学步车**”提供最简单的平地 and 最低的速度要求。随着训练步数的增加，如果你观察到这两个指标 **像阶梯一样稳步上升**，这说明你的强化学习模型正在健康收敛，机器人正在不断“通关”更难的挑战；反之，如果它们卡在低等级上不去，往往说明机器人在平地慢走时就遇到了瓶颈（例如陷入了某种局部最优或者频繁摔倒），无法满足升级条件。

1.2.3 bad_orientation 初值说明

1. bad_orientation 为 1.0 说明了什么？
这个指标代表了 **回合因为“姿态错误”而提前结束的比例**。limit_angle: 0.8（弧度，约等于 45.8 度）意味着只要机器人的躯干发生严重的倾斜（如前倾、后仰或侧翻超过近 45 度），系统就会判定机器人“摔倒了”，从而强制终止当前的回合 (Episode)。在训练初期，它的值为 1.0（即 100%），这说明了：**机器人每一次尝试都以“严重倾斜/摔倒”告终**。因为在训练刚开始时，神经网络的参数是随机初始化的，机器人输出的关节指令是混乱且无逻辑的。它就像一个失去小脑控制的婴儿，一开机就会因为四肢乱动而瞬间失去平衡并摔倒在地。因此，所有回合无一例外都是因为触发了 bad_orientation 而被强行重置的。
2. 随着训练推进，这个值应该发生什么变化？
随着强化学算法不断试错并迭代优化网络权重，机器人会逐渐掌握保持平衡和走路的技巧。因此，这个指标的变化趋势应当是：
 - 迅速下降并趋近于 0**：随着机器人学会了直立并能稳定走动，因为失去平衡而摔倒的次数会越来越少，bad_orientation 会从 1.0 快速下降。
 - time_out 显著上升**：你会看到另一个指标 Episode_Termination/time_out 开始上升并趋近于 1.0。time_out 代表机器人成功存活到了设定的最大回合时间（在你的配置中，episode_length_s: 20.0，即完美存活 20 秒）。这是最理想的终止情况。
3. **总结**：初期为 1.0 是完全正常的“婴儿摔跤期”。如果训练推进了很久（比如几千次迭代后），这个值依然居高不下（比如一直在 0.5 以上），那通常意味着模型遇到了瓶颈（例如动作空间配置错误或奖励权重冲突），导致机器人始终学不会稳定的平衡。

1.3 奖励函数源码分析

操作手册中给出了 3 个奖励函数的源码（upward、energy、foot_clearance_reward），回答以下问题：

1. 选择其中 1 个函数，用自己的话解释它的工作原理（输入是什么、计算逻辑是什么、输出代表什么）；

2. foot_clearance_reward 中使用了 tanh 函数，它的作用是什么？为什么脚着地时不会误罚支撑脚？

1. energy 的工作原理说明：
 - 输入：
 - env (ManagerBasedRLEnv)：当前的强化学习环境对象，包含了物理引擎中所有并行的环境数据和机器人状态。
 - asset_cfg (SceneEntityCfg)：指定要计算哪个资产（这里默认是 "robot" 即机器人本身）以及包含哪些关节（joint_ids）。
 - 计算逻辑：
 - 获取关节速度**：qvel = asset.data.joint_vel 获取当前机器人所有关节的角速度 (Angular Velocity)。
 - 获取关节力矩**：qfric = asset.data.applied_torque 获取当前施加在所有关节上的输出力矩 (Torque/Force)。
 - 计算绝对功率**：分别对速度和力矩取绝对值 torch.abs()，然后将它们逐元素相乘。在物理学中，**功率 (Power) = 力矩 (Torque) × 角速度 (Velocity)**。取绝对值是为了防止在做负功（例如电机作为发电机刹车时）时被抵消，因为不管是正转还是反转，只要输出力矩产生了运动，就在消耗电池电量。
 - 求和**：使用 torch.sum(..., dim=-1) 将单个机器人的所有关节（通常是 29 个自由度）的功率消耗加在一起，得到该机器人此刻的总能耗。
 - 输出代表什么 (Output)：
 - 输出一个一维张量 (torch.Tensor)，形状为 [num_envs]（环境数量，例如 4096），即同时返回 4096 个并行环境中每个机器人的 **瞬时总机械功率消耗估值**。
 - 因为在 env.yaml 中这个 reward 项被赋予了负的权重 (-2.0e-05)，所以这个输出值将被用作 **惩罚项**。输出值越大（即机器人消耗的瞬时功率越高，比如用极大的力气快速挥动手臂），受到的惩罚就越重。这迫使强化学习模型去寻找最“省电”、最柔和且高效的行走步态。
2. foot_clearance_reward 中 tanh 函数充当了一个 **基于速度的平滑门控 (Gating/Masking) 函数**。它计算的是脚在水平面 (XY 平面) 的运动速度。tanh 能够将脚的水平速度平滑地映射到 [0, 1) 的区间内：
 - 当脚在 **快速水平移动** 时（速度大），tanh 的输出非常接近 1。
 - 当脚 **静止不动** 时（速度接近 0），tanh 的输出非常接近 0。
3. 为什么脚着地时不会误罚支撑脚？
强化学习系统要求“摆动脚”（Swinging foot）抬高到 target_height（比如 0.1 米）来跨越障碍。但是，“支撑脚”（Stance foot）在踩在地面上承重时，高度必定接近 0，与 target_height 有巨大的误差。如果直接惩罚这个误差，机器人就根本没法踩地了。
不误罚的奥秘就在于这一行：

```
reward = foot_z_target_error * foot_velocity_tanh
```

- 对于摆动脚（在空中跨步）**：水平速度很大，tanh ≈ 1。此时公式变为 reward = foot_z_target_error * 1。高度误差被全额保留。如果脚没有抬到指定高度，reward 会很大，最后经过 exp(-reward) 处理后得到一个极低的分数，从而 **强制要求移动的脚步必须抬高**。
- 对于支撑脚（踩在地面上）**：支撑脚紧贴地面，不发生相对滑动，因此水平速度 ≈ 0，tanh ≈ 0。此时公式变为 reward = foot_z_target_error * 0 = 0。无论此时脚和目标高度的误差有多大，乘以 0 之后都会被完全屏蔽/抹除。最后 exp(0) = 1，支撑脚直接获得满分奖励。
- 总结**：tanh 巧妙地通过“水平速度”来区分出脚当前是处于“摆动期”还是“支撑期”。只有当脚在水平移动时（摆动期），算法才会对它的抬腿高度进行要求考核；一旦脚踩在地上不动了（支撑期），考核自动关闭，从而完美避免了对支撑脚的误罚。

1.4 Sim2Sim 概念理解

1. 用一段话解释：什么是 Sim2Sim？为什么不直接把 Isaac Lab 训练的模型部署到真机上，而要先在 MuJoCo 中验证？

2. 用一段话解释：为什么 Sim2Sim 验证选择 MuJoCo 而不是 Gazebo？从物理引擎的接触模型、运行速度、API 设计等角度分析；

3. ONNX 格式的作用是什么？为什么不直接用 PyTorch 的 .pt 文件进行部署？

1. Sim2Sim（仿真到仿真转录）是指将强化学习算法在一个物理引擎（如 Isaac Lab 使用的 PhysX）中训练出的模型，放入另一个计算逻辑完全不同的物理引擎（如 MuJoCo）中进行交叉验证的过程。不直接将 Isaac Lab 模型部署到真机上的核心原因在于“物理引擎漏洞利用（Simulator Exploitation）”：由于 Isaac Lab 追求 GPU 大规模并行训练，其接触动力学和数值求解往往不够精确，AI 极易学到利用这些 PhysX 特有的物理“Bug”来维持平衡的奇葩动作。如果直接上真机，这些违背真实物理规律的动作会瞬间导致机器人失控摔倒甚至损毁昂贵的硬件。而 MuJoCo 以高精度的多体动力学和接触计算闻名，将模型放入 MuJoCo 就像一个“照妖镜”，只有能在另一套底层物理规则下依然保持稳健行走的策略，才具备真实的鲁棒性，从而在确保安全的前提下大幅提高部署到物理真机（Sim2Real）的成功率。
2. 在 Sim2Sim 验证环节中选择 MuJoCo 而非 Gazebo，主要归结于其在底层计算和架构上的三大优势：首先，[在接触模型上](#)，Gazebo 默认的 ODE 引擎处理刚体碰撞时极易产生高频抖动和数值爆炸，而 MuJoCo 采用了独特的软约束（Soft-Constraint）和凸优化数学公式来处理多体动力学与接触摩擦，能够极其精确、平滑且稳定地模拟足式机器人的复杂触地反力，是最接近真实世界连续物理特性的引擎之一；其次，[在运行速度上](#)，Gazebo 携带着庞大的 ROS 通信框架和渲染前端历史包袱，运行效率低下，而 MuJoCo 是用纯 C 语言编写的极致精简引擎，单 CPU 核即可实现远超实时（数千倍）的仿真速度，完美契合强化学习高频试错和快速评估的需求；最后，[在 API 设计上](#)，Gazebo 依赖异步的发布/订阅（Pub-Sub）机制，这会带来不可控的通信延迟和状态同步问题，导致 RL 需要的“状态 - 动作”步进循环难以对齐，而 MuJoCo 提供了透明、无状态且直接的 Python/C API，开发者可以零延迟地同步读取和写入所有底层的矩阵数据与关节力矩，使其成为连接神经网络模型和物理评估环境的最佳桥梁。
3. [ONNX \(Open Neural Network Exchange, 开放神经网络交换\)](#) 是一种用于表示机器学习模型的开放标准格式。你可以把它理解为 AI 模型界的“PDF 格式”——无论你是用 Word (PyTorch) 还是 PPT (TensorFlow) 创建的内容，都可以导出为统一的 PDF (ONNX)，从而在任何设备上无缝打开和查看。

在机器人控制等工程领域，不直接使用 PyTorch 的 `.pt` 文件而选择转换为 ONNX 进行部署，主要是出于以下几个致命痛点：

1. 摆脱沉重的环境依赖 (轻量化部署)
- `.pt` 的劣势**：`.pt` 文件深度绑定了 Python 语言和 PyTorch 框架。如果你要在真机（例如机器狗主板）上运行它，通常需要安装几 GB 大小的 PyTorch 环境和整个 Python 解释器。对于算力和存储空间寸土寸金的嵌入式设备来说，这极其臃肿且难以维护。
 - ONNX 的优势**：ONNX 是独立于框架的计算图表示。导出为 ONNX 后，你可以使用极其轻量级的 C++ 推理引擎（如 ONNX Runtime）直接加载运行，[完全脱离 Python 和 PyTorch](#)。
2. 追求极致的实时性与低延迟 (性能优化)
- `.pt` 的劣势**：Python 本身自带全局解释器锁 (GIL)，且 PyTorch 在前向推理时有很多为了训练而保留的额外开销（动态图计算）。对于像四足机器人这样要求 [500Hz 甚至 1000Hz \(即 1 毫秒内完成计算\)](#) 的高频控制循环，Python 的运行延迟和抖动是致命的。
 - ONNX 的优势**：ONNX 采用静态计算图。各大硬件厂商（如 NVIDIA 的 TensorRT、Intel 的 OpenVINO、ARM 的 NPU 工具链）都针对 ONNX 格式做了极其深度的底层硬件加速。你可以用纯 C++ 零延迟地调用 ONNX 模型，执行速度往往比 `.pt` 原生推理快数倍甚至数十倍。
3. 语言与生态的兼容性 (C++ 友好)
- 真正的工业级机器人底层飞控和电机驱动几乎全是用 [C/C++](#) 编写的。虽然 PyTorch 提供了 C++ 版本的 LibTorch 来读取 `.pt`，但 LibTorch 依然极其庞大复杂，编译耗时且极易产生版本冲突。相比之下，ONNX Runtime 的 C++ 接口非常干净、稳定，能够极其顺滑地嵌入到机器人原有的 C++ 工程代码中。
4. **总结**：在实验室里做训练和验证时，我们用 `.pt` 因为它方便调试；但当模型要上真机“干活”时，转换为 ONNX 则是为了实现 [去 Python 化、极致加速和工业级的 C++ 嵌入部署](#)。

1.5 Aelos 机器人基础操作

1. 阅读 SMART 基础使用教程，了解 Aelos 机器人的启动流程和安全注意事项；
2. 安装示教软件，完成与机器人的连接；
3. 启动机器人，通过示教软件完成至少 2 个简单的动作（如站立、挥手、蹲下等）；
4. 提供以下验证材料：
 - 机器人启动截图 1 张；
 - 示教软件界面截图 1 张；
 - 机器人执行动作的照片。

1.6 Aelos 机器人自由度分析

1. 分析 Aelos 的自由度（DoF）分布；
2. 基于 Aelos 的自由度分析，思考以下问题：
 - 你想利用 Aelos 完成哪些类型的任务？举例说明；
 - 你觉得 Aelos 目前最难以完成的任务是什么？为什么？

基础作业提交要求

- 📎

请将以上内容整理为 [一份 PDF](#)，包含必要的截图说明和问题的回答。

参考

环境：Isaac Sim + Isaac Lab + MuJoCo 参考资料：

1. [APLaS-Plus/unitree_rl_lab](#)
2. [unitreerobotics/unitree_mujoco](#)
3. [google-deepmind/mujoco: Multi-Joint dynamics with Contact. A general purpose physics simulator.](#)
4. [unitreerobotics/unitree_sdk2: Unitree robot sdk version 2. https://support.unitree.com/home/zh/developer](#)